Student Name	Student ID
Raghad Hatim Hakami	445014026

Repsitory Link : https://github.com/raghadHatim/LibraryBookTracker.git

Parent of all Custom Exceptions Classes:

```
public class BookCatalogException extends Exception {

public BookCatalogException (String r){
    super(r);
}

}
```

Children of `BookCatalogException` `Class` :

```
public class InsufficientArgumentsException extends BookCatalogException{

  public InsufficientArgumentsException (String m ){
   super(m);
  }
}
```

```
public class InvalidFileNameException extends BookCatalogException {

    public InvalidFileNameException(String m){
        super(m);
    }
}
```

```
public class MalformedBookEntryException extends BookCatalogException {

  public MalformedBookEntryException(String m){
    super(m);
  }

}
```

```
public class InvalidISBNException extends BookCatalogException {

    public InvalidISBNException (String m){
     super(m);
    }
```

```
}
```

```
public class DuplicateISBNException extends BookCatalogException {

    public DuplicateISBNException(String m){
        super(m);
    }
}
```

Organization Class to implement Encapsulation :

```
public class LibraryOrganization {
    private String title ;
    private String author ;
    private String isbn ;
    private int copies ;

    public LibraryOrganization(String title, String author, String isbn, int
copies) {
        this.title = title;
        this.author = author;
        this.isbn = isbn;
        this.copies = copies;
    }

    public String getTitle(){
     return this.title;
    }
     public String getAuthor(){
     return this.author;
    }
     public String getIsbn(){
     return this.isbn;
    }
     public int getCopies(){
     return this.copies;
    }

    @Override
    public String toString() {

        return String.format("%-30s %-20s %-15s %5d", title, author, isbn,
copies);
    }

    //Implement Structured Text input
```

```
    public String toFileFormat() {
        return title + ":" + author + ":" + isbn + ":" + copies;
    }
}
```

LibraryBookTracker class that contain main methods:

```
import java.io.*;
import java.time.LocalDateTime;
import java.util.*;
public class LibraryBookTracker {
    // Statistics count (Declear with static becuse it's belong to class and
out of the main )
    static int validRecords = 0;
    static int searchResults = 0;
    static int booksAdded = 0;
    static int errorCount = 0;
    public static void main(String[] args) {
        String logFile = "errors.log"; //log file name

        try {
            //should take 2 argument
            if (args.length < 2) {
                throw new InsufficientArgumentsException("Missing
arguments.");
            }
            String fileName = args[0]; //file name
            //if first argument do not end with .txt throw error(catch on
main)
            //endWith is prebuild method in java compare the end of string
with suffix we deciede
            if (!fileName.endsWith(".txt")) {
                throw new InvalidFileNameException("File must end with .txt");
            }
            File file = new File(fileName); // if no file exist
            //getParent return string
            if (file.getParent() != null) {
                logFile = file.getParent() + "/errors.log"; // if the parent
folder exist java will build the file inside it
            } else {
                logFile = "errors.log"; //else java will add the file at the
same folder we work in it
            }
            // getParentFile return instance file
            if (file.getParentFile() != null) { // if file not exist this
function will help to create it (file)
                file.getParentFile().mkdirs(); // and mkdirs function will
keep it in same order and create it
            }
```

```
            file.createNewFile(); //if already exist will not do anything
            String userInput = args[1];  //operation name
            // 13 - ISBN Digit
            if (isNumeric(userInput) || userInput.length() == 13) {
                if (userInput.length() == 13 && isNumeric(userInput)) {
                    searchBooks(file, userInput, true, logFile);
                } else {
                    throw new InvalidISBNException("ISBN must be 13 digits and
numeric: " + userInput);
                }
            // New Book Record
            } else if (userInput.contains(":")) {
                addBook(file, userInput, logFile);
            // New Title Keyword
            } else {
                searchBooks(file, userInput, false, logFile);
            }
        // i keep logical order custom first then more general ones
        } catch (BookCatalogException e) {
            String errorData = "No Argument";
            if (args.length > 1) {
                errorData = args[1]; //if error with ISBN
            }
            logError(logFile, errorData, e);
            System.out.println("Error: " + e.getMessage());
        // tohandle file errors
        } catch (IOException e) {
            String errorData = "No Argument";
            if (args.length > 1) {
                errorData = args[1];
            }
            logError(logFile, errorData, e);
            System.out.println("File System Error: " + e.getMessage());
        } finally {
            System.out.println("\n--- Statistics ---");
            System.out.println("Valid records processed: " + validRecords);
            System.out.println("Search results found: " + searchResults);
            System.out.println("Books added: " + booksAdded);
            System.out.println("Errors encountered: " + errorCount);
            System.out.println("Thank you for using The Library book
Tracker");
        }
    }
/*
@param from args[0] file object
@param from args[1] operation String
@param true if lloking for ISBN and false if looking for Title
```

```java
@param log file to record errors
@return nothing void type
@exception DuplicateISBNException deal with it locally by print statment
@IOException because i open and read from file so i could hadle any exception
accure in the main method
*/
    private static void searchBooks(File f, String q, boolean isIsbn, String
log) throws IOException {
        ArrayList<LibraryOrganization> results = new ArrayList<>();
        Scanner reader = new Scanner(f); //t o read file

        while (reader.hasNextLine()) { // to read it line by line after check
            String line = reader.nextLine(); //store line from file as string
            try {
                LibraryOrganization book = parsing(line); // parsing the line
and store it as an object
                validRecords++; // if no error found increase the validRecords

                boolean isFound = false;
                //if is ISBN operation
                if (isIsbn) {
                    isFound = book.getIsbn().equals(q);
                //if is Title operation
                } else {
                    if
(book.getTitle().toLowerCase().contains(q.toLowerCase())) {
                        isFound = true;
                    }
                }
                //if found will add it to result and print it to user
                if (isFound) {
                    results.add(book);
                }
                //this will catch all parcing method exceptions
            } catch (BookCatalogException e) {
                logError(log, line, e);
            }
        }
        reader.close(); //close scanner
        //handle dublicate ISBN
        if (isIsbn && results.size() > 1) {
            DuplicateISBNException d = new DuplicateISBNException("More than
one book with ISBN: " + q);
            logError(log, q, d);
            System.out.println("Error: " + d.getMessage()); //print error here
            return;
        }
```

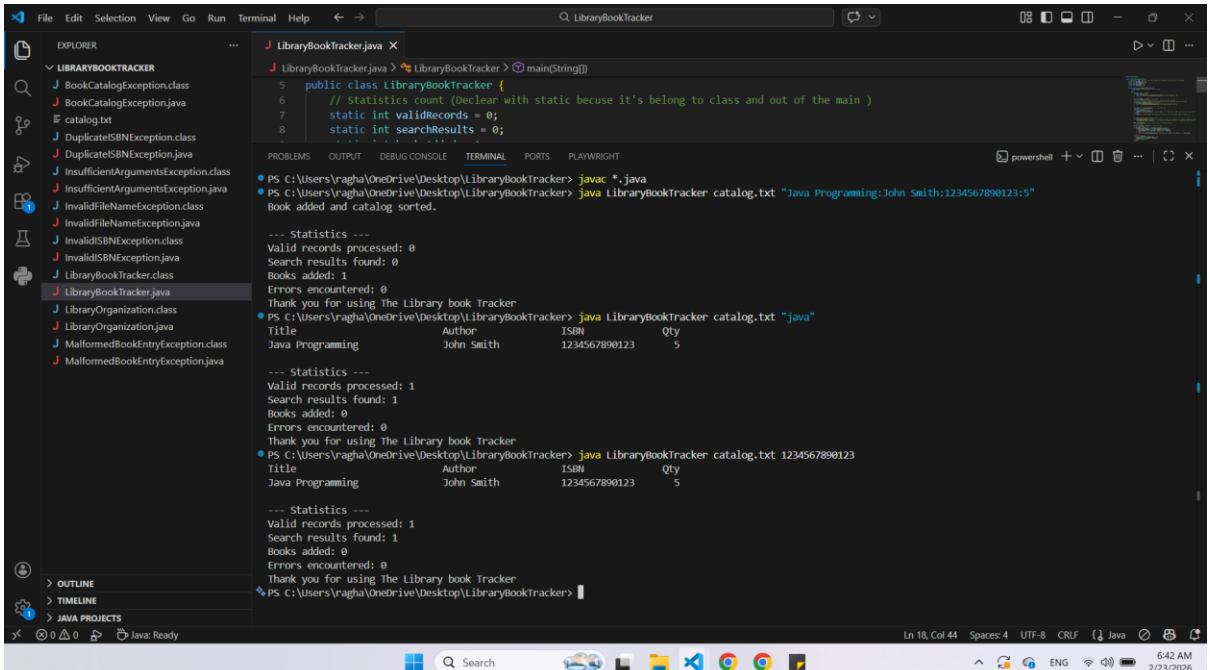
```java
        if (results.isEmpty()) {
            System.out.println("No results found.");
        } else {
            System.out.printf("%-30s %-20s %-15s %5s\n", "Title", "Author",
"ISBN", "Qty");
            for (int i = 0; i < results.size(); i++) {
                System.out.println(results.get(i));
                searchResults++;
            }
        }
    }
/*
@param from args[0] file object
@param from args[1] operation String if added full structured text
@param log file to record errors
@return nothing void type
@IOException because i open and read from file so i could hadle any exception
accure in the main method
*/
        private static void addBook(File f, String data, String log) throws
IOException {
        ArrayList<String> Lines = new ArrayList<>();
        //Read file and transfer file lines to Lines list
        Scanner reader = new Scanner(f);
        while (reader.hasNextLine()) {
            Lines.add(reader.nextLine());
        }
        reader.close();

        try {
            LibraryOrganization newBook = parsing(data);
            Lines.add(newBook.toFileFormat()); //add book with required format
            //Sort file
            Collections.sort(Lines, String.CASE_INSENSITIVE_ORDER);
            //Rewrite to file again all information
            PrintWriter writer = new PrintWriter(f);
            for (String s : Lines) {
                writer.println(s);
            }
            writer.close();

            System.out.println("Book added and catalog sorted.");
            booksAdded++;
            // catch parsing method error
        } catch (BookCatalogException e) {
            logError(log, data, e);
```

```
            System.err.println("Error adding book: " + e.getMessage());
        }
    }
/*
@param The structured text that follow this shape AB : CD : EF : GH;
@return Object from LibraryOrganization class
@exception Validate it is contain 4 parts;
@exception Validate Title & Author not empty ;
@exception Validate ISBN is all numbers and exactly 13 digit;
@exception Validate Copies exactly greater than 0;
*/
    private static LibraryOrganization parsing(String text) throws
BookCatalogException {
        String[] parts = text.split(":"); //this is prebuild in java method
that spilt words depend on value of parameter
        //Throw Exceptions if found one
        // all this Exceptions catche on SearchBook & AddBook methods
        if (parts.length != 4) {
            throw new MalformedBookEntryException("Invalid format (need 4
fields)");
        }
        if (parts[0].isEmpty() || parts[1].isEmpty()) {
            throw new MalformedBookEntryException("Title or Author empty");
        }
        if (parts[2].length() != 13 || isNumeric(parts[2]) == false) {
            throw new InvalidISBNException("ISBN must be 13 digits");
        }

        try {
            int num = Integer.parseInt(parts[3]); // prebuild java method turn
String to Integer
            if (num <= 0) {
                throw new MalformedBookEntryException("Copies must be > 0");
            }
            return new LibraryOrganization(parts[0], parts[1], parts[2], num);
            // This catch for this line int num = Integer.parseInt(parts[3]);
becuse parseInt force us to deal with Exception
        } catch (NumberFormatException e) {
            throw new MalformedBookEntryException("Copies must be a number");
        }
    }
/*
@param file name
@param string type that trrigir error
@param object for Exception
@return nothing void type
@exception IOException
```

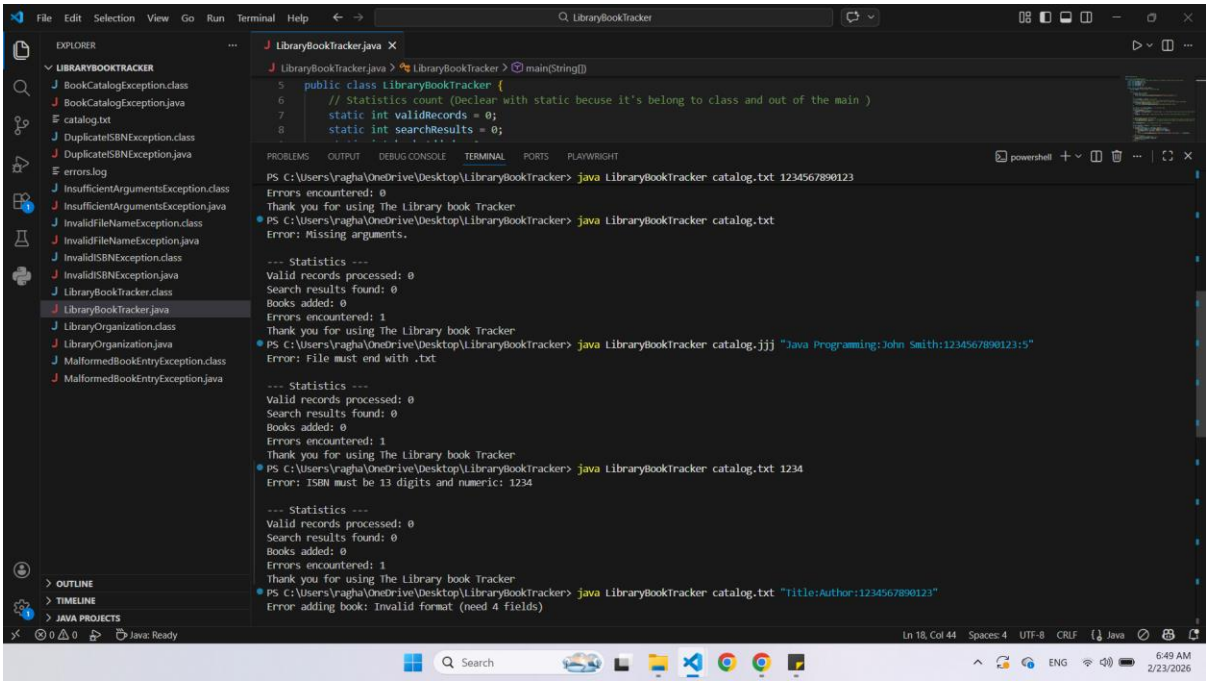
```java
*/
    private static void logError(String path, String info, Exception e) {
        errorCount++;
        String time = LocalDateTime.now().toString(); //use prebuild time
library in java

        try {
            FileWriter f = new FileWriter(path, true); // open file without
delete preivous information
            PrintWriter p = new PrintWriter(f); // to allow me use println
            p.println("[" + time + "] " + e.toString() + " - Input: \"" + info
+ "\""); // record error
            p.close();
        } catch (IOException io) {
            System.out.println("Could not write to log file.");
        }
    }
/*
@param Take String type to check if just contain number or not
@return true or false
*/
    public static boolean isNumeric(String s) {
        if (s == null || s.length() == 0) return false;
        for (int i = 0; i < s.length(); i++) {
            if (!Character.isDigit(s.charAt(i))) {
                return false;
            }
        }
        return true;
    }
}
```

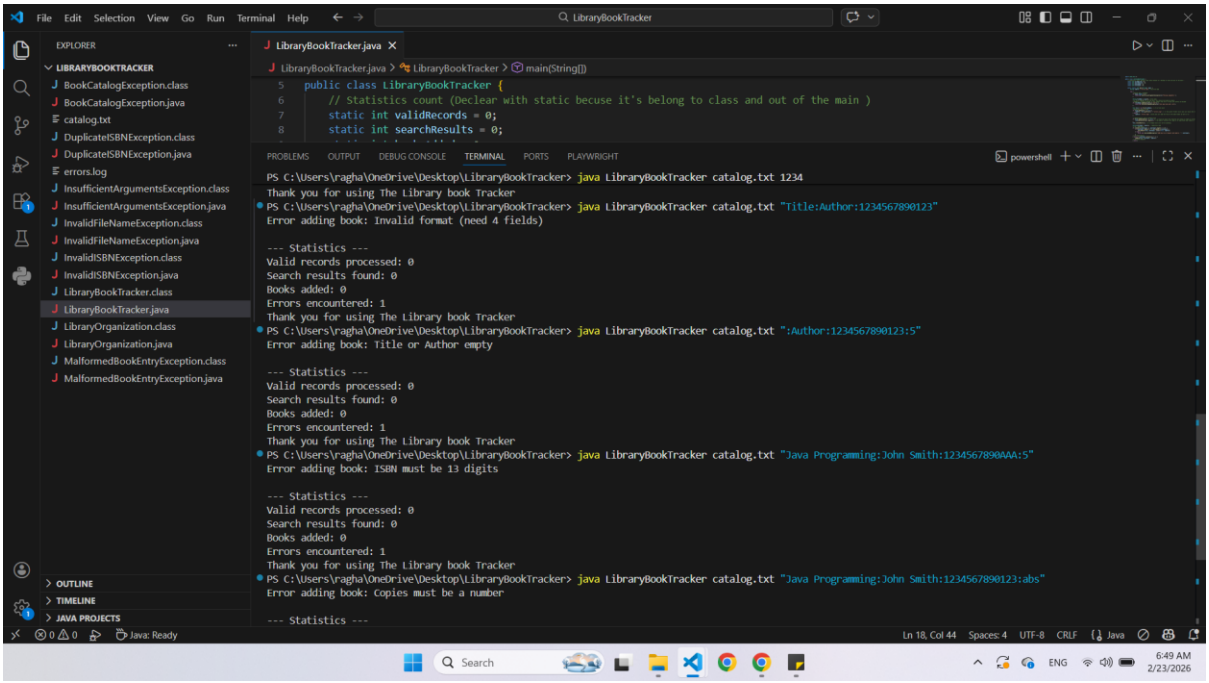
Screenshots :

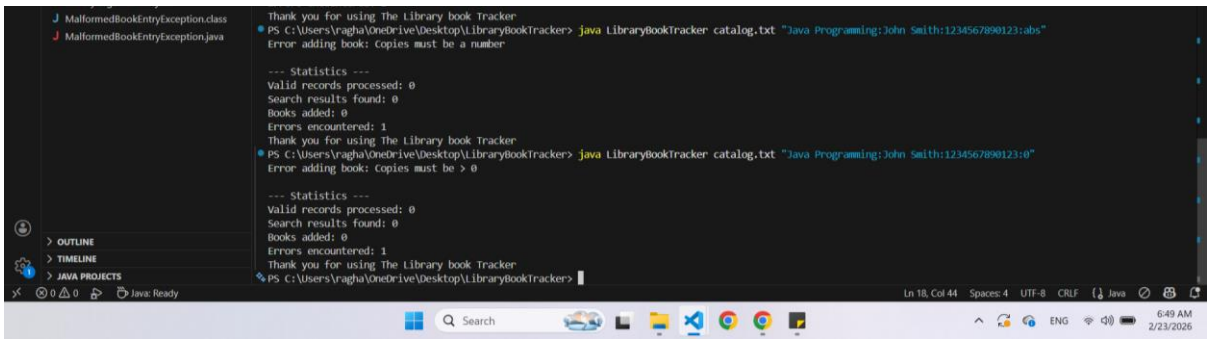
Valid Cases :

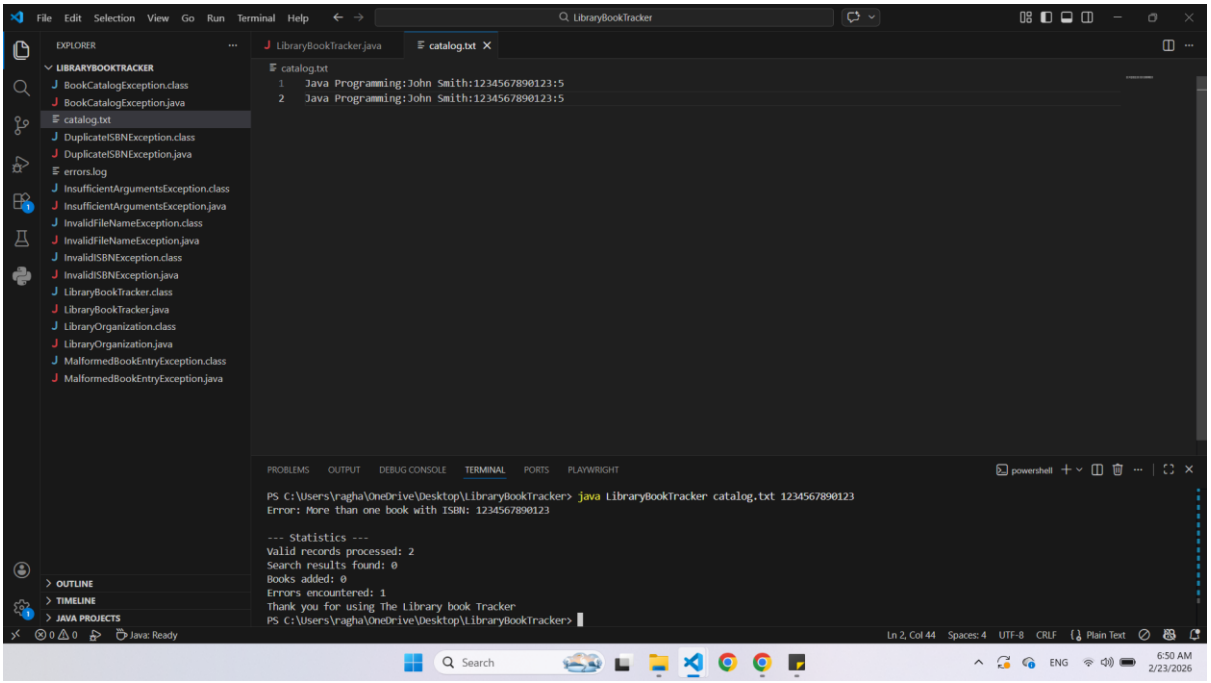
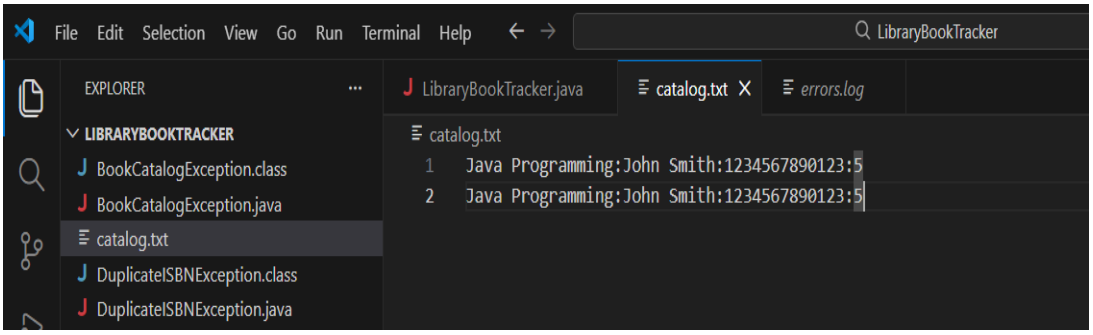
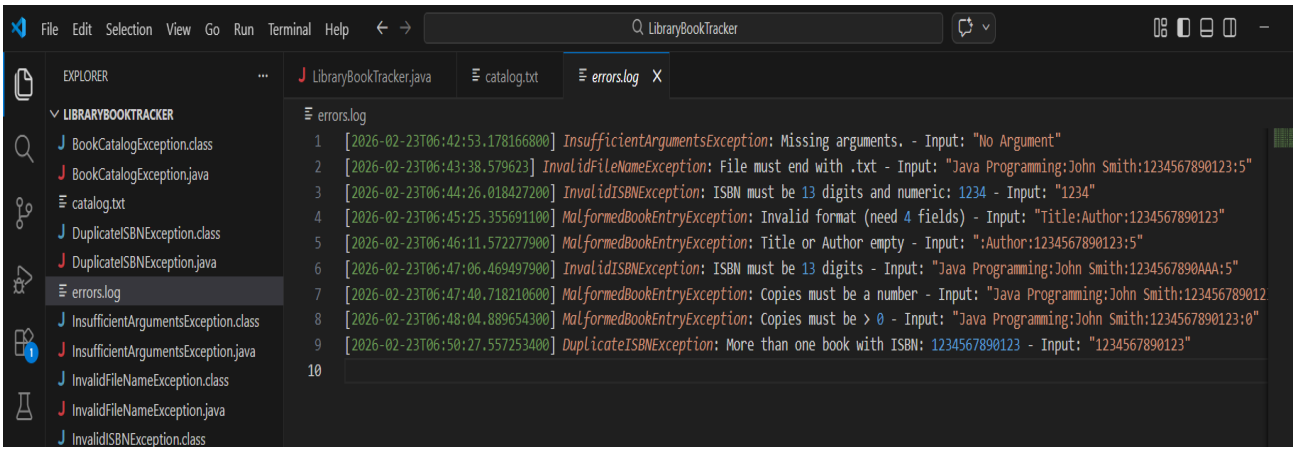


Invalid Cases :









Catalog.txt File & Log File